Task 10 – Theory Questions

Generic Programming, Delegates & Lambda Expressions

Part 01 – Theory Questions

Q1: What are the benefits of using a generic sorting algorithm over a non-generic one?
A generic sorting algorithm operates on any data type without duplicating code. It provides compile-time type safety, which prevents type mismatch errors at runtime. Code reuse is maximized because a single generic method replaces multiple type-specific implementations. Performance is also improved since boxing and unboxing of value types is avoided. Maintenance becomes easier as changes are applied in one place rather than across multiple overloads.

Q2: How do lambda expressions improve the readability and flexibility of sorting methods?
Lambda expressions allow sorting criteria to be defined inline at the call site, removing the need for separate named methods. This makes the code shorter and easier to follow because the logic is visible exactly where it is used. Flexibility is increased because the comparison logic can be changed per call without modifying the sorting method itself. This enables developers to sort the same collection in different ways simply by passing different lambda expressions.

Q3: Why is it important to use a dynamic comparer function when sorting objects of various data types?
Different data types have different notions of order. A string may be sorted alphabetically, by length, or case-insensitively, while an object may be sorted by one of several properties. A dynamic comparer function separates the sorting algorithm from the comparison logic, making the method truly reusable. Without a dynamic comparer, a new sorting method would have to be written for every type and every ordering criterion, leading to code duplication and reduced maintainability.

Q4: How does implementing IComparable<T> in derived classes enable custom sorting?
IComparable<T> defines a standard contract that requires the class to implement a CompareTo method. When a derived class implements this interface, sorting algorithms can call CompareTo without knowing the specific type at compile time. This enables polymorphic behavior where each class defines its own natural ordering. For Manager objects, this means salary-based comparison is built into the class itself, so any generic sort that works with IComparable<T> will automatically use the correct logic.

Q5: What is the advantage of using built-in delegates like Func<T, T, TResult> in generic programming?
Built-in delegates eliminate the need to define custom delegate types for common patterns. Func is already part of the .NET framework, so it is immediately recognizable to other developers. It integrates seamlessly with lambda expressions and LINQ. Using Func also reduces the amount of boilerplate code and keeps the codebase consistent. In generic programming, Func allows methods to accept behavior as a parameter without tightly coupling the method to any specific implementation.

Q6: How does the usage of anonymous functions differ from lambda expressions in terms of readability and efficiency?

Anonymous functions use the delegate keyword followed by a parameter list and a block body, which makes them more verbose but explicit. Lambda expressions use the => syntax and are more concise, especially for single-expression logic. Both compile to the same IL code, so there is no runtime performance difference. However, lambda expressions are preferred in modern C# because they are easier to read and write. Anonymous functions are occasionally used when a block body with multiple statements is needed without the overhead of a named method.

Q7: Why is the use of generic methods beneficial when creating utility functions like Swap?

A generic Swap method works with any data type, eliminating the need for separate swap methods for integers, strings, objects, and so on. Type safety is preserved because the compiler enforces that both elements being swapped belong to the same type. The method is also more readable and self-documenting than using temporary variables with explicit casting. Generic utility methods are building blocks for larger generic algorithms, making the entire codebase more consistent and reusable.

Q8: What are the challenges and benefits of implementing multi-criteria sorting logic in generic methods?

The main challenge is designing a comparer that correctly handles all combinations of primary and secondary criteria without introducing bugs in the tie-breaking logic. The comparer must return zero only when all criteria are equal, which requires careful ordering of comparisons. The benefit is that a single sort pass produces a fully ordered result without requiring multiple sorts. It also keeps the sorting logic centralized, making it easier to adjust priorities. Generic multi-criteria sorting is particularly valuable when working with complex business objects that have several meaningful orderings.

Q9: Why is the default(T) keyword crucial in generic programming, and how does it handle value and reference types differently?

default(T) provides a safe way to initialize generic variables without knowing the concrete type at compile time. For value types such as int or double, it returns zero or the equivalent zero-value. For reference types such as classes or strings, it returns null. Without default(T), a generic method cannot safely assign an initial value to a variable of type T because the syntax for value and reference type initialization differs. This makes default(T) essential for writing correct generic algorithms that handle both categories of types uniformly.

Q10: How do constraints in generic programming ensure type safety and improve the reliability of generic methods?

Constraints restrict the types that can be used as type arguments, ensuring that the generic method only receives types that support the required operations. For example, the ICloneable constraint guarantees that Clone() can be called on any instance of T, preventing runtime exceptions. Constraints also enable the compiler to provide IntelliSense and type checking based on the guaranteed interface. This moves potential errors from runtime to compile time, making code more reliable. Constraints communicate intent clearly, serving as documentation about what capabilities T must have.

Q11: What are the benefits of using delegates for string transformations in a functional programming style?

Delegates allow transformation logic to be passed as a parameter, making functions higher-order and composable. The same ApplyTransform method can convert strings to uppercase, reverse them, or apply any other transformation simply by passing a different delegate. This avoids writing a separate

method for each transformation and encourages separation of concerns. In a functional style, data flows through a pipeline of transformations, each represented by a delegate. This results in concise, testable, and reusable code that is easy to extend with new transformations.

Q12: How does the use of delegates promote code reusability and flexibility in implementing mathematical operations?

A method that accepts a delegate for its operation can perform addition, subtraction, multiplication, or any other operation without being rewritten. The operation is injected at the call site, decoupling the method from any specific computation. This follows the Open/Closed principle: the method is open for extension through new delegates but closed for modification. It also makes unit testing easier because each operation can be tested independently. Delegates effectively turn methods into first-class values that can be stored, passed, and invoked like any other variable.

Q13: What are the advantages of using generic delegates in transforming data structures?

Generic delegates allow a single transform method to convert collections of any input type to any output type. This eliminates the need for type-specific conversion methods and reduces code duplication significantly. The compiler enforces type correctness, preventing accidental mismatches between input and output types. Generic delegates also integrate naturally with LINQ and functional patterns in C#. They make APIs more expressive because the signature clearly communicates both the input and output types of the transformation.

Q14: How does Func simplify the creation and usage of delegates in C#?

Func is a built-in generic delegate type that covers the most common delegate patterns, eliminating the need to declare custom delegate types. It supports up to 16 input parameters and one return value, making it flexible enough for most scenarios. Because Func is part of the standard library, it is immediately understood by other developers without additional documentation. It works directly with lambda expressions, enabling concise inline definitions. Using Func reduces boilerplate and keeps code focused on logic rather than infrastructure.

Q15: Why is Action preferred for operations that do not return values?

Action explicitly signals that a delegate performs a side effect rather than computing a value. This makes the intent of the code clearer to readers. Using Func<T, void> is not valid in C#, so Action fills this role specifically. It supports up to 16 parameters and integrates with lambda expressions just like Func. Choosing Action over a custom void delegate keeps the code consistent with .NET conventions, improves readability, and avoids unnecessary custom type definitions.

Q16: What role do predicates play in functional programming, and how do they enhance code clarity?

A predicate is a function that returns a boolean, representing a condition or test. In functional programming, predicates are used to filter, partition, and validate data without embedding the condition inside a loop or method. Predicate<T> makes the filtering criterion explicit and reusable. Code clarity improves because the condition is named or expressed inline at the point of use, making it immediately clear what is being filtered. Predicates also compose well, allowing complex conditions to be built from simpler ones.

Q17: How do anonymous functions improve code modularity and customization?

Anonymous functions allow behavior to be defined exactly where it is needed without polluting the class with small helper methods. This keeps related logic together and reduces the cognitive load of

navigating between method definitions. Customization is easy because each call site can provide its own logic without affecting others. Anonymous functions capture local variables through closures, allowing them to adapt their behavior based on the surrounding context. This makes them well-suited for event handling, callbacks, and short-lived transformation tasks.

Q18: When should you prefer anonymous functions over named methods in implementing mathematical operations?

Anonymous functions are preferred when the logic is simple, used in only one place, and does not need to be tested or reused independently. They keep the code concise by avoiding unnecessary method declarations. If the operation is used in multiple places or is complex enough to warrant testing, a named method is more appropriate. Anonymous functions are also useful when the operation depends on local variables that would otherwise need to be passed as extra parameters to a named method.

Q19: What makes lambda expressions an essential feature in modern C# programming?

Lambda expressions provide a concise syntax for defining inline functions, which is fundamental to LINQ, asynchronous programming, and functional patterns in C#. They remove the verbosity of anonymous delegates and named methods for simple operations. Lambdas integrate with the type system through expression trees, enabling frameworks like Entity Framework to translate them into SQL queries. They improve readability by keeping logic close to where it is used, and they enable a declarative programming style that focuses on what to compute rather than how to compute it.

Q20: How do lambda expressions enhance the expressiveness of mathematical computations in C#?

Lambda expressions allow mathematical operations to be described in a form that closely resembles mathematical notation, making code easier to understand. Complex operations like exponentiation or division can be passed as arguments without defining separate methods. This enables generic numeric algorithms that accept any operation as a parameter. The concise syntax reduces noise so the reader can focus on the mathematical relationship being expressed. Lambdas also support closures, allowing operations to capture constants or configuration values from the enclosing scope.

Part 02 – Research Topics

1. LinkedIn Article – Delegates in Implementing the Functional Paradigm

Title: Delegates and the Functional Paradigm in C# – Writing Cleaner, More Flexible Code

In modern software development, the ability to treat functions as first-class citizens is no longer exclusive to languages like Haskell or F#. C# has embraced functional programming concepts through delegates, and the results are transformative.

A delegate is a type-safe reference to a method. It allows methods to be passed as parameters, stored in variables, and invoked dynamically. This is the foundation of higher-order functions – a core concept in functional programming.

The .NET framework ships with three powerful built-in delegate types: **Func**, which encapsulates a method that returns a value; **Action**, for methods that produce side effects without returning a value; and **Predicate**, for methods that evaluate a condition and return a boolean. Together, these delegates enable a declarative, composable style of programming.

Consider a list of employee records. Using a Func delegate with a lambda expression, we can sort by salary, filter by department, or transform names – all without modifying the underlying data structure. The behavior is injected at the call site, making each operation explicit and independently testable.

Lambda expressions are the syntax that brings delegates to life. Instead of defining a named method, a lambda lets you express the logic inline: (x) => x * x is both readable and compact. Combined with LINQ, delegates and lambdas form a powerful pipeline for data transformation.

The functional approach with delegates also improves maintainability. When behavior is passed as a parameter rather than hardcoded, changing requirements become a matter of passing a different delegate rather than rewriting methods. This aligns with the Open/Closed Principle and reduces technical debt.

In conclusion, delegates are the bridge between object-oriented and functional paradigms in C#. Mastering them is essential for any developer who wants to write flexible, clean, and modern code.

2. Research Topics

Parallel Programming and Concurrency

Parallel programming involves dividing a task into subtasks that execute simultaneously across multiple processor cores. In C#, this is achieved using the Task Parallel Library (TPL), the Parallel class, and PLINQ (Parallel LINQ). Concurrency is a broader concept that refers to multiple tasks making progress over the same time period, not necessarily executing at the exact same instant.

Key constructs include: Parallel.For and Parallel.ForEach for data parallelism; Task.Run for offloading work to thread pool threads; ConcurrentCollections such as ConcurrentDictionary and ConcurrentBag for thread-safe data access; and synchronization primitives like lock, Mutex, and SemaphoreSlim for managing shared state.

The primary challenge in concurrent programming is avoiding race conditions, deadlocks, and data corruption. These issues arise when multiple threads access shared resources without proper coordination. Modern C# encourages immutability and lock-free designs to minimize these risks.

Unit Testing and Test-Driven Development (TDD)

Unit testing is the practice of writing automated tests that verify individual units of code – typically methods or classes – in isolation. The most widely used testing framework in the C# ecosystem is xUnit, alongside NUnit and MSTest. Tests are written as methods decorated with [Fact] or [Test] attributes and use assertion libraries to verify expected outcomes.

Test-Driven Development (TDD) is a development methodology that reverses the traditional write-then-test order. The TDD cycle consists of three steps known as Red-Green-Refactor: first, write a failing test that describes the desired behavior (Red); then, write the minimum code needed to make the test pass (Green); finally, refactor the implementation while keeping all tests passing (Refactor).

TDD produces code that is inherently testable because design decisions are driven by how the code will be used and verified. Benefits include faster defect detection, better documentation through tests, and higher confidence when refactoring. Mocking frameworks such as Moq allow dependencies to be replaced with controlled substitutes, enabling true unit-level isolation.

Asynchronous Programming with async and await

Asynchronous programming allows a program to initiate a long-running operation – such as a network request or file I/O – and continue executing other work while waiting for the result. In C#, this

is implemented using the async and await keywords, which build on the Task-based Asynchronous Pattern (TAP).

A method marked with async returns a Task or Task<T>. Inside the method, the await keyword suspends execution until the awaited Task completes, freeing the calling thread to perform other work. This avoids blocking threads, which is critical for maintaining UI responsiveness in desktop applications and maximizing throughput in web servers.

Best practices include: always awaiting async methods rather than calling .Result or .Wait(), which can cause deadlocks; using ConfigureAwait(false) in library code to avoid capturing the synchronization context; and using CancellationToken to support cooperative cancellation. The async/await pattern integrates naturally with Entity Framework, HttpClient, and all modern .NET I/O APIs.

Part 03 – Bonus

1. Self-Study Report

Overview

This self-study report reflects on the learning journey undertaken during the exploration of generic programming and delegates in C#. The primary goal was to understand how these features contribute to writing clean, reusable, and maintainable code.

Topics Studied

The study covered generic classes and methods, type constraints, the IComparable<T> and ICloneable interfaces, the SortingAlgorithm<T> and SortingTwo<T> patterns, delegate types, Func, Action, Predicate, anonymous functions, and lambda expressions.

Key Learnings

Generics eliminate the need for type-specific code duplication while preserving type safety. Constraints provide a mechanism to guarantee that type arguments support the operations required by the algorithm. Delegates enable a functional style of programming where behavior is treated as data. Lambda expressions dramatically reduce boilerplate, making intent clearer and code shorter.

Challenges Encountered

The most significant challenge was understanding when to use IComparable<T> versus a separate Func comparer. Over time it became clear that IComparable encodes the natural ordering of a type, while Func comparers are better suited for context-specific orderings. Another challenge was the subtle difference between anonymous functions and lambda expressions, which appear similar but have different syntax and historical context.

Reflection

Working through twenty coding problems in a single task reinforced how interconnected these concepts are. Generics, delegates, and lambdas are not isolated features; they work together to enable the expressive, declarative programming style that characterizes modern C#. This study has provided a solid foundation for approaching more advanced topics such as LINQ, asynchronous programming, and reactive extensions.

2. What is Asynchronous Programming?

Asynchronous programming is a programming model that allows a program to perform multiple operations concurrently without blocking the execution thread. Rather than waiting idly for a slow operation to complete, the program can initiate the operation and continue processing other tasks, resuming work on the original operation once its result becomes available.

In traditional synchronous code, each statement must complete before the next one begins. This is acceptable for fast operations but becomes a serious problem for I/O-bound tasks such as reading files, querying databases, or making HTTP requests. Blocking a thread while waiting for a network response wastes resources and reduces the scalability of the application.

C# addresses this through the async and await keywords. When a method is marked async, it can contain await expressions that pause execution at that point and yield control back to the caller. The method resumes automatically when the awaited operation completes. Crucially, the thread is not blocked during the wait – it is free to handle other work.

A practical example is a web API handler. Without async, each request would hold a thread while waiting for the database query to return. With async/await, the same thread can begin processing a second request while the first waits for its database result. This dramatically increases throughput without requiring additional hardware.

Asynchronous programming is especially important in: desktop applications, where blocking the UI thread freezes the interface; web servers, where thousands of concurrent requests must be handled efficiently; and mobile applications, where battery and network constraints make non-blocking I/O essential.

Common best practices in C# async programming include: always awaiting Task-returning methods instead of using blocking calls like .Result; propagating cancellation tokens through the call chain; handling exceptions with try/catch around await expressions; and using Task.WhenAll when multiple independent async operations can run in parallel. Mastering these practices leads to applications that are responsive, scalable, and efficient.